Generating Clothing Images with a Conditional Deep Convolutional Generative Adversarial Network

Generative AI Applications Analysis Report

Author: Francisco Mateus

Dataset: Fashion-MNIST (Xiao et al., 2017)

Overview

This project trained a conditional Deep Convolutional Generative Adversarial Network (DCGAN) to generate `28x28` grayscale clothing images from the Fashion-MNIST dataset (Xiao et al., 2017), with the model conditioned on one of the ten garment labels. This involved training the generator and discriminator pair, plus a separate small classifier that evaluates whether generated images match their requested labels. For the conditional result, the classifier assigned the requested label to `0.9008` of generated images, vs `0.9074` on real test images. The two scores sat close together, showing that the labels largely control the generated content. These numbers say the image shows the right type of item; they do not say the image is realistic.

Dataset or Prompt Description

The data used in this project comes from the Fashion-MNIST dataset (Xiao et al., 2017), available at https://github.com/zalandoresearch/fashion-mnist. This dataset contains grayscale images of clothing products with class annotations. Below are its details:

- total images: `70,000`
- class labels: `10` garment categories (T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot)
- training images: `60,000` (`6,000` per class)
- test images: `10,000` (`1,000` per class)
- image size: `28x28` pixels, grayscale
- license: MIT, attributed to the upstream Fashion-MNIST repository, as recorded in `DATASET.md`

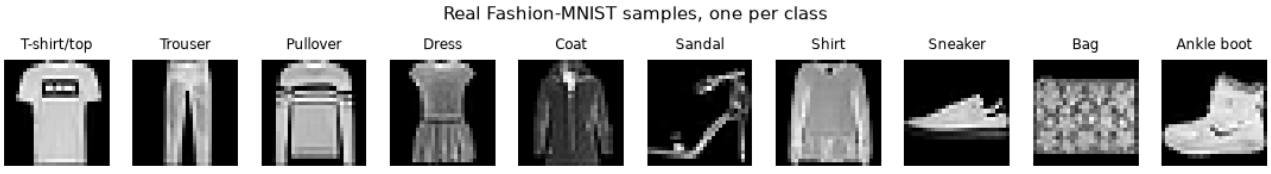


Figure 1: one real Fashion-MNIST sample from each of the ten classes.

To generate an image for a requested class, the model has to get the structure that makes that class what it is: outline, contrast, shading. This was challenging because several classes share silhouettes and fine detail at `28x28` grayscale: the evaluation classifier most often confused real T-shirt/top images with Shirt, and the same ambiguity appeared in the generated samples, as discussed in the evaluation section. The ten categories remain recognisable at this resolution, which makes the dataset practical for visual inspection of generated samples.

The training images were divided between the project's two models. The generator and discriminator trained on the full `60,000`-image training set. The evaluation classifier used a stratified pool of `5,000` images per class, meaning equal numbers of each class, holding out the remaining `1,000` images per class:

- classifier training: `50,000` images, used to update classifier weights
- classifier validation: `10,000` images, used to monitor per-epoch accuracy (no gradient updates, no model selection)
- test set: the original `10,000`-image test set, held out from training; used only after classifier training (test accuracy and Fréchet features), never for model selection

Data handling and parameters used in the project:

- pixel values scaled to the range `[-1, 1]`, matching the generator's `Tanh` output
- offline-first loading from the repository's `data/` directory, with the notebook verifying the four archive checksums against `DATASET.md` before training (`download=True` documented fallback only)
- random seed: `42`, used for the split, training, and generation

Model Design and Training Approach

Generator and discriminator architecture

Both networks were designed with reference to the DCGAN guidelines proposed by Radford et al. (2016), which describes architectural choices intended to support stable GAN training. The implementation adopted these principles with one deliberate architectural deviation; this deviation is mentioned later in the discriminator section.

The generator starts from a latent vector of 100 random numbers, one per image. A dense projection reshapes this vector into a 7x7 feature grid. Two transposed convolutions (`ConvTranspose2d`, kernel 4, stride 2) then double the spatial size at each step, from 7x7 to 14x14 to 28x28. A final Tanh layer maps the pixel values into the [-1, 1] range, matching the normalized input data.

Batch normalization follows the generator's dense projection and both upsampling layers. In the discriminator, it follows the second downsampling convolution. It stabilizes training by normalizing activations as they pass through each network. Following the DCGAN guidelines, it is omitted from the generator's output layer and the discriminator's first layer.

In the discriminator, strided convolutions reduce the image resolution at each stage, and LeakyReLU activations with a negative slope of 0.2 let small gradients pass through negative inputs, as recommended by the DCGAN guidelines.

Conditional generation

Following Mirza and Osindero (2014), I made the GAN conditional so that it generates images of a requested Fashion-MNIST category, rather than producing unlabelled samples from the 10 categories at random. Class labels are supplied to both networks: the generator uses the label to shape what it produces, and the discriminator uses it to judge whether an image matches its requested class, as described below.

In the generator, the requested class label is mapped to a 16-dimensional learned embedding: a vector of numeric values that represents the class. This gives the model a trainable representation of each category, rather than a one-hot encoding, which identifies classes but does not capture relationships between them. The embedding dimension was fixed at 16 and not tuned. Instead, the ablation budget was spent on the label-smoothing variant, so each experiment changed only one factor.

The label embedding is concatenated with the 100-dimensional noise vector, giving a 116-dimensional input. A dense layer projects this input into a `7x7x256` feature volume (12,544 values), which forms the starting point for image generation. The layer has `1,467,648` parameters (116 × 12,544 weights plus 12,544 biases), about 68.3% of the generator's total `2,149,409` parameters.

In the discriminator, a separate label embedding maps each class to a 28x28 grid of values. This grid is concatenated with the image as a second input channel, so the convolutional layers process the image and its label together from the first layer. This allows the

discriminator to judge not only whether an image looks real, but also whether it matches the class it was requested for. Because the generator is penalised when its output fails this check, it receives a training signal to respect the requested label.

The one deliberate deviation from the DCGAN guidelines is in the discriminator's output layer. After two downsampling stages, the feature map has `128` channels at a spatial resolution of `7x7`. Rather than applying a further convolution, the model flattens this feature map and passes it through a single fully connected layer, `Linear(128*7*7, 1)`, which maps its `6,272` values to one real-or-fake score. Each value therefore has its own learned weight in the final decision.

This dense output head was chosen over a further convolutional layer because it maps the final feature representation directly to a single real-or-fake score. The layer has `6,272` weights and one bias parameter. Since this departs from the convolutional output layer recommended in the DCGAN guidelines, it is recorded here as a deliberate architectural deviation.

Training configuration

Training alternates between two updates, both scored with `BCEWithLogitsLoss` applied to raw logits, the numerically stable form of the standard yes/no loss. In the discriminator update, the network is trained on a batch of real images with a smoothed target of `0.9` and a batch of generated images with a smoothed target of `0.1`. In the generator update, the generator is trained to make the discriminator score its images as real, using the hard target of `1`.

I used two-sided label smoothing, with targets of `0.9` for real images and `0.1` for generated images, to keep the discriminator from becoming overconfident and weakening the learning signal available to the generator. Salimans et al. (2016, Section 3.4) instead recommend one-sided smoothing, in which only the real target is smoothed and the generated target remains at `0`. Both approaches are reasonable, so I compared them empirically rather than assuming either would perform better. The ablation results are presented in the evaluation section.

Both networks were trained using Adam with a learning rate of `2e-4` and betas of `(0.5, 0.999)`, following the optimizer settings recommended by Radford et al. (2016). Training took approximately `8.9` seconds per epoch.

Beyond the headline run, I trained three additional models under the same configuration, each for 50 epochs. Two used different random seeds (43 and 44) to assess sensitivity to initialisation, and one used one-sided label smoothing to isolate the effect of that choice. The generator loss flattens from epoch 47 onward, so 50 epochs appears sufficient for it to stabilise.

Parameter	Configuration
Model	Conditional DCGAN
Dataset	Fashion-MNIST
Epochs	50
Batch size	128
Latent dimension	100
Label embedding	16
Optimizer	Adam
Learning rate	0.0002
Betas	(0.5, 0.999)
Random seed	42
Sensitivity seeds	43 and 44
Smoothing (headline run)	two-sided (0.9 / 0.1)
Hardware	NVIDIA GeForce RTX 4080 Laptop
Training time	444 seconds

BatchNorm and the generation protocol

Generating images in training mode raises a consistency problem. In training mode, the generator's BatchNorm layers compute statistics from the current batch, so the same noise vector and class label can produce different pixel values depending on which other samples are in the batch.

To measure this effect, I generated images from the same fixed noise vectors and class labels under six combinations of model mode and batch composition. Five combinations produced results, each using 1,000 generated tiles. However, the sixth combination, which combined training mode with a batch size of 1, failed. The generator applies `BatchNorm1d`

to the projected feature vector before reshaping it into a spatial volume. I believe that in training mode, this layer needs more than one sample to compute batch statistics, so PyTorch rejects the forward pass when the batch contains a single sample.

The batch-size failure shows that training-mode generation depends on batch composition and cannot produce single samples, so I adopted evaluation mode for all image generation. In this mode, BatchNorm uses the running statistics accumulated during training rather than statistics calculated from the current batch, which removes the batch-composition dependence observed in the training-mode tests. Every generated sample in the notebook, including the failure strip, uses this setting. The measured comparison between the two modes is reported in the generation protocol results subsection.

Loss interpretation

The discriminator and generator losses use different numbers of BCE terms, so their chance baselines also differ:

- **Discriminator:** A coin-flip prediction gives an expected combined loss of `2 × ln(2) ≈ 1.386` across its two BCE terms.
- **Generator:** Its single BCE term gives a chance baseline of `ln(2) ≈ 0.693`.

The loss figure includes both baselines.

Output Evaluation and Interpretation

Evaluation overview

The notebook produces seven figures; six of them are in this section, and the real-image reference strip sits in the dataset section.

Measurement	Result	What it evaluates
Generated classifier agreement	0.9008	Class agreement
Real test agreement	0.9074	Classifier agreement on real images
Generated Fréchet distance	21.0	Distribution difference in classifier feature space
Real-vs-real reference distance	1.3	Reference distance in the same feature space
Macro diversity ratio	0.87	Generated versus real intra-class diversity
Agreement across seeds 42/43/44	0.9008 / 0.9020 / 0.8843	Seed sensitivity of the headline run
One-sided smoothing agreement	0.9100	Ablation variant

Training behaviour

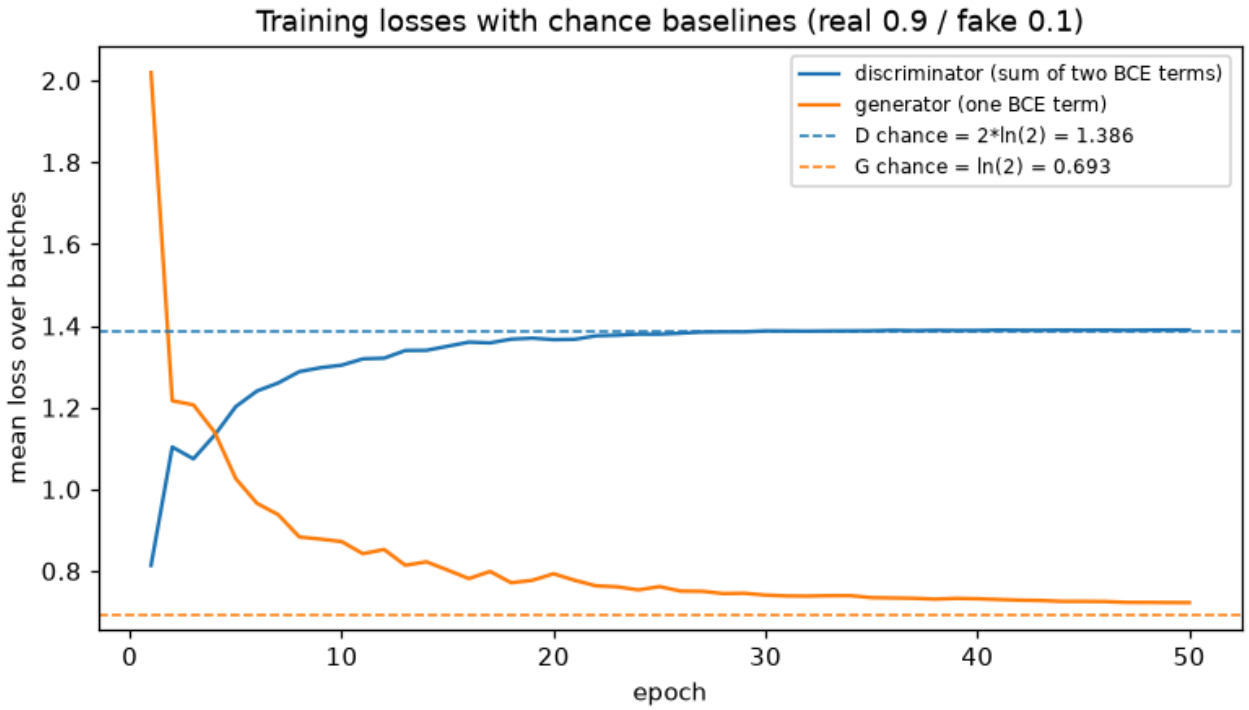


Figure 2: mean discriminator and generator losses per epoch, with their chance baselines.

The loss curves show how training progressed under the selected label-smoothing approach. The discriminator loss started at `0.814` after the first epoch, below its `1.386` chance baseline. It then increased to `1.30` at epoch 10 and `1.37` at epoch 18. During the final twenty epochs, it remained between `1.387` and `1.390`, close to and slightly above chance.

The generator loss fell from `2.019` to `0.872` at epoch 10. It then approached its `0.693` chance baseline, reaching `0.723`. The loss remained at approximately this level from epoch 47 onward.

By the end, both losses were close to their respective chance baselines. However, near-chance losses do not establish that the two networks reached a stable equilibrium or that the generated images match the real data distribution.

Qualitative evaluation

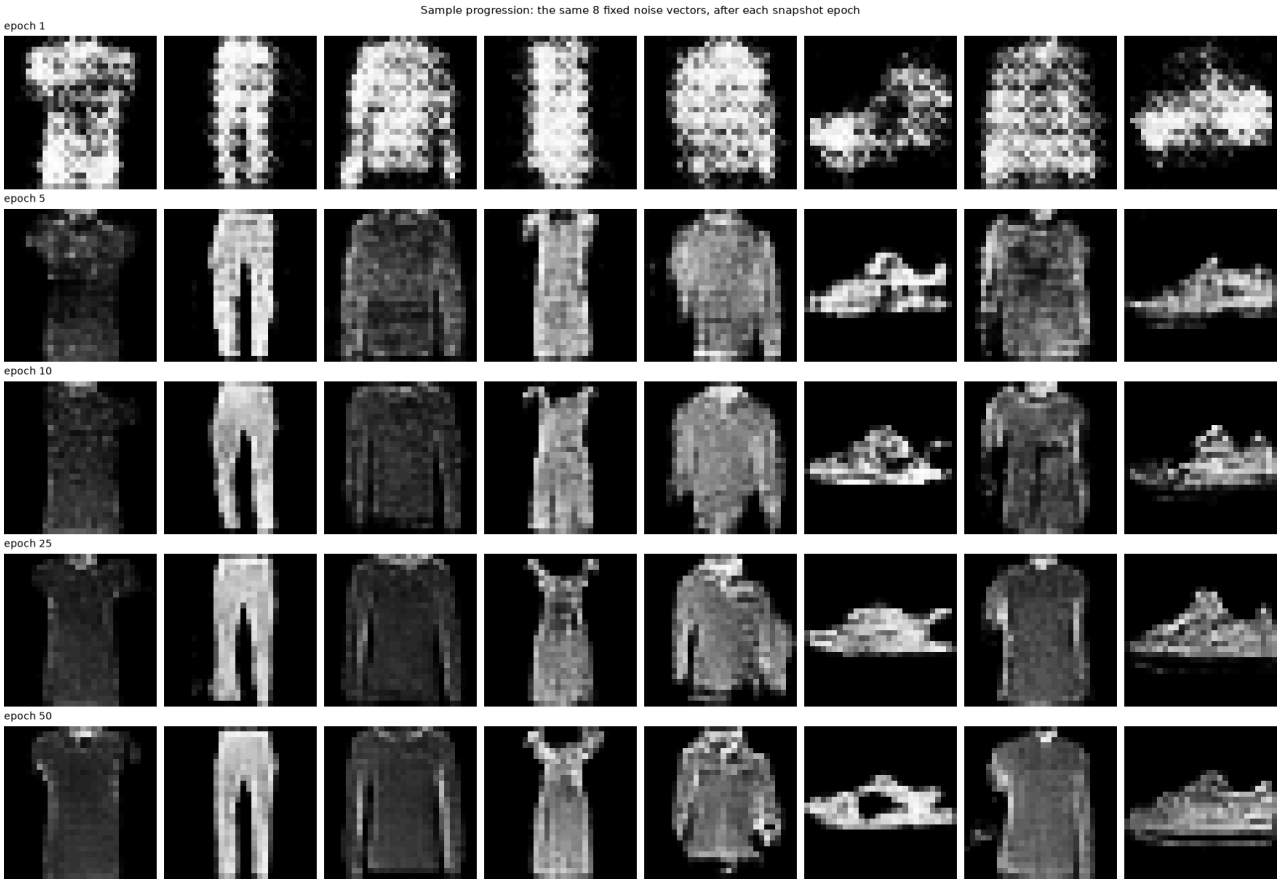


Figure 3: the same eight fixed (noise, label) pairs, rendered after snapshot epochs 1, 5, 10, 25, and 50.

At epoch 1, the tiles mainly show noise with faint garment outlines. By epoch 5, trousers and a sandal silhouette are recognisable. From epochs 10 to 50, collars, soles, and sleeves become more defined. The same samples retain their identities across the snapshots. Later epochs appear to refine the existing shapes and details rather than substantially change the identity of the generated samples.



Figure 4: class-conditioned grid. Each column represents one fixed noise vector, while the label changes down the rows.

The conditional grid provides the clearest illustration of class control. Each row shows the requested class across all eight noise columns. The Trouser row contains trousers, the Bag row contains bags, and the Sandal row shows heel sandals. The Sneaker row shows low sneakers, while the Ankle boot row shows boots.

Across a row, the noise changes while the requested class remains fixed: shading and detail vary between samples while the overall garment category is preserved. Down the rows, the label changes and the generated garment generally follows. This illustrates how a conditional GAN uses class information alongside random noise to control image generation (Mirza & Osindero, 2014).

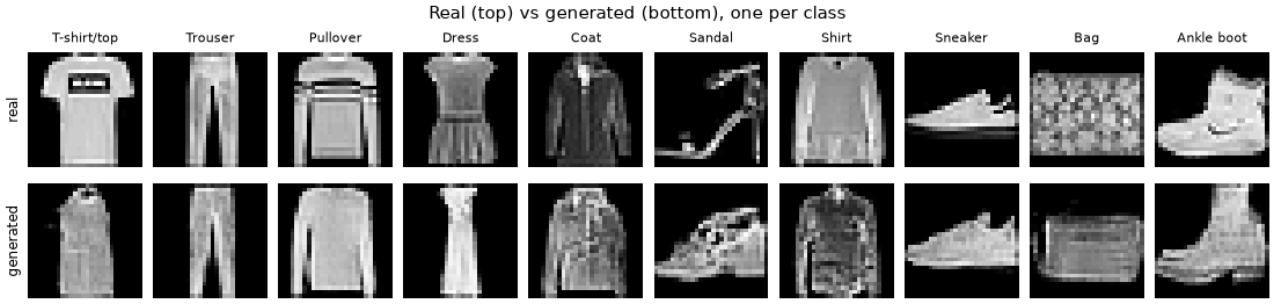


Figure 5: one real (top) and one generated (bottom) sample per class.

In these examples, the generated silhouettes retain recognisable features of their requested classes, such as two trouser legs, the heel under the sandal, and the shaft of the boot. The visual quality gap is also apparent: the generated images are grainier, have flatter grayscale values, and contain less fine texture than the real images.

Class agreement

The grids give the visual side of the evaluation. For a quantitative check, I also measured class agreement using a separate classifier, following Odena et al. (2017), who ask whether generated images are recognised as their intended classes.

I trained a small CNN on a stratified pool of `50,000` images drawn from the real training split. It has two convolution blocks, a dense head, and `421,834` parameters.

Validation accuracy, measured on a held-out split of `10,000` real images, was tracked after each epoch. It reached `0.9019` after 1 epoch and `0.9199` after 10 epochs, and ranged from `0.917` to `0.925` over the last five epochs. This plateau suggests validation performance has

stabilised, although ten epochs and one seed do not establish convergence. The test split was evaluated once, after training: the classifier read `0.9074` of the `10,000` test images correctly.

I then generated `1,000` images per class (`10,000` in total) with the trained generator, using evaluation seed `43` and the selected evaluation-mode protocol. The classifier scored each generated image against its requested class. I call this measurement *class agreement*: does a classifier trained on real data read the image as the class that was requested? It is a narrow check. It says nothing about whether an image is realistic or varied, which the following measurements address.

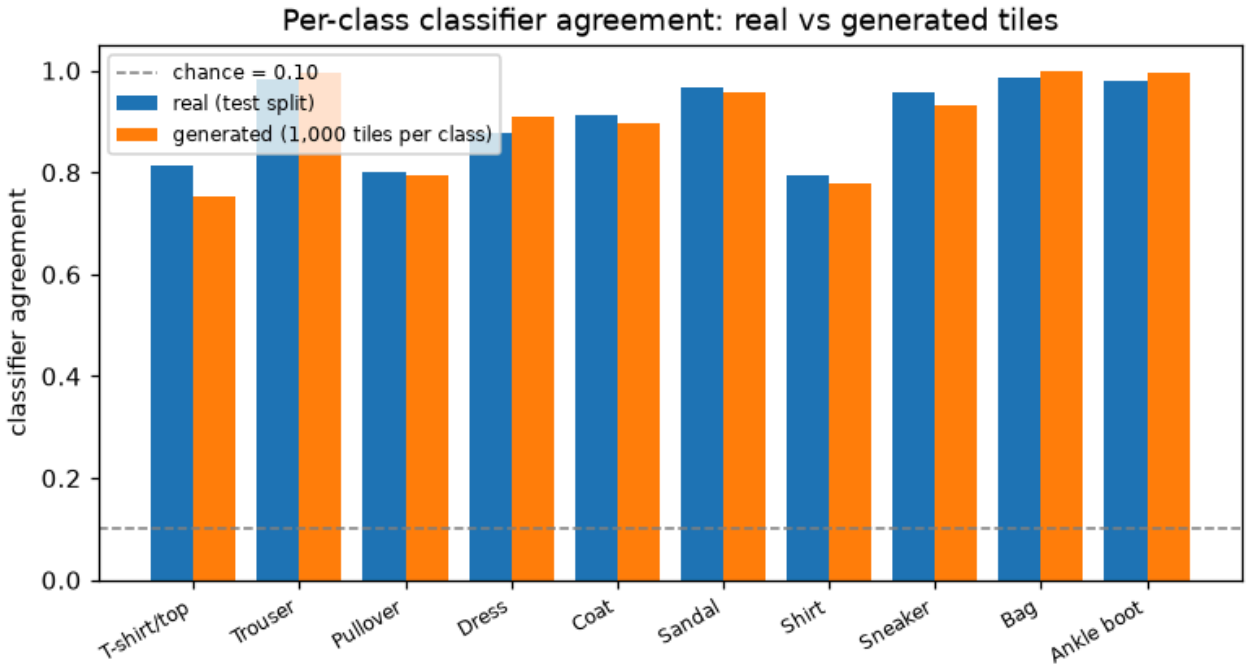


Figure 6: Per-class classifier agreement for real test images and generated images. The dashed line represents chance agreement (`0.10`).

The classifier assigned the requested label to 0.9008 of generated images. Its accuracy on the real test split was 0.9074, so the generated images scored 0.0066 below the real ones. This difference was small relative to the approximate sampling variability at these sample sizes. The per-class results are:

Class	Generated	Real test
T-shirt/top	0.75	0.81
Trouser	1.00	0.98
Pullover	0.80	0.80
Dress	0.91	0.88
Coat	0.90	0.91
Sandal	0.96	0.97
Shirt	0.78	0.79
Sneaker	0.93	0.96
Bag	1.00	0.98
Ankle boot	1.00	0.98

T-shirt/top had the largest negative difference: 0.75 for generated images against 0.81 for real images, a gap of 0.06. Given a standard error of about 0.02 for the difference between two estimates at 1,000 images each, this gap is larger than sampling noise alone would explain.

The confusion breakdowns show where the errors go. For the 1,000 real T-shirt/top test images, the classifier misclassified 187. The most common error was Shirt (135 images, or 0.14 of the class), followed by Pullover (0.02) and Dress (0.02). For the 1,000 generated T-shirt/top images, the classifier recognised 752 correctly and misclassified the remaining 248. The most common error was again Shirt (167 images, or 0.17), followed by Dress (0.04) and Pullover (0.03).

Both breakdowns therefore share the same main confusion, with Shirt first, but the generated images show a higher Shirt error rate (0.17 against 0.14). Real T-shirt/top images were already hard for the classifier, scoring 0.81 on the real test split. The evidence does not show whether the extra Shirt errors come from the generator or from the classifier's decision boundary. The pattern is consistent with ambiguity already present in the dataset, but it does not rule out that the generator adds its own weaknesses.

Distribution similarity

Class agreement does not show how closely the generated images match the real ones as a whole, or how much they vary within each class. Two further measurements address these gaps: distribution similarity and intra-class diversity.

The first measurement is the Fréchet distance in the classifier's feature space. It compares the real and generated image distributions using the classifier's internal representation of each image: the `128`-dimensional activations of the penultimate layer, the last layer before the class scores. Following Heusel et al. (2017), I fitted a Gaussian to each feature distribution, summarised by its mean and covariance, and computed the squared Fréchet distance between the two Gaussians. The real distribution contained the `10,000` test images, and the generated distribution contained `10,000` generated images. The resulting squared Fréchet distance was `20.9619`.

I also calculated a real-vs-real comparison to establish a reference scale. This applied the same calculation to the real test split and the held-out validation split of `10,000` real images. It produced a distance of `1.3127`. Both sides were unseen by the classifier's training and both contained exactly `1,000` images per class, providing a reference with equal sample sizes and class balance.

The generated distribution's **squared Fréchet distance** (`20.9619`) was about sixteen times the real-vs-real reference (`1.3127`), so the generated images sit measurably farther from the real images in the classifier's feature space. These absolute values are specific to this feature space and are not directly comparable with Inception-based results in the literature. The figure should therefore be read as a within-project comparison, not a general measure of image quality.

The gap also varies by class. Per-class distances range from `19.7376` for Trouser, where agreement is `1.00`, to `77.4557` for Bag, where agreement is also `1.00`. A class can thus score well on agreement while its generated images remain far from the real ones in feature space. Class agreement and Fréchet distance measure different things, so both are needed to judge the generator.

Intra-class diversity

The second measurement is intra-class diversity: how much the generated images of one class vary among themselves compared with the real images of that class. For each class, I calculated the mean pairwise squared feature distance among the `1,000` generated images

and among the 1,000 validation images, then took the ratio of generated to real. Ratios ranged from 0.77 for T-shirt/top and Dress to 0.98 for Sandal, and the ratio of the macro-averaged distances was 0.87.

A ratio below 1 means the generated images are less spread out than the real ones. Generated images are therefore somewhat less diverse overall, with the largest shortfalls in T-shirt/top and Dress and almost none in Sandal. This statistic cannot confirm full coverage: a ratio near 1 does not show that every variation in the real data is reproduced, so partial mode collapse cannot be ruled out.

Generation protocol results

In training mode, the generator's class agreement depends strongly on batch composition: it fell from 0.8830 with mixed-class batches to 0.1070 with single-class batches. The same image also varied between these two compositions, with a maximum pixel difference of 1.9997. The batch-size-one protocol could not be tested in training mode, because BatchNorm needs more than one image to calculate its statistics.

In evaluation mode, all three batch compositions gave the same class agreement of 0.8980, and the maximum pixel difference between mixed-class and single-class batches was exactly 0.0000. Changing the batch size from 128 to 1 still produced a small difference of about 0.0022, so outputs are identical across batch compositions but not strictly across batch sizes.

The practical conclusion is that mixed-class batches alone do not guarantee consistent generation, since training-mode outputs still vary with batch composition. Generating in evaluation mode is what removes this dependence on composition.

Seed sensitivity

The headline measurements come from a single training run with seed `42`. To test how much the results depend on the starting seed, I repeated the full `50`-epoch training with seeds `43` and `44`, changing nothing else. Each generator produced `1,000` images per class, and all three sets were scored by the same classifier, which was trained once and kept fixed, using the same evaluation protocol.

Seed	Macro agreement	Final discriminator loss	Final generator loss
42	0.9008	1.390	0.723
43	0.9020	1.387	0.728
44	0.8843	1.387	0.732
Spread	**0.0177**	0.003	0.009

Class agreement varied by 0.0177 across three seeds, and the losses varied very little. This is a small sample of seeds, so it suggests but does not establish that class agreement is insensitive to the seed. The Fréchet distance and intra-class diversity were not repeated across seeds, so their seed sensitivity remains untested.

Label-smoothing ablation

The adopted training setup uses two-sided label smoothing, with a real target of `0.9` and a generated target of `0.1`. To compare against an alternative, I ran one additional training with the one-sided variant from the training configuration, keeping the same seed and changing nothing else.

The one-sided variant scored better on both metrics: a macro agreement of `0.9100` (versus `0.9008`) and a squared feature-space Fréchet distance of `16.1506` (versus `20.9619`). Because this comparison rests on a single extra run, it is not enough to conclude that the one-sided variant is consistently better. For that reason, the two-sided variant remains the adopted configuration, and the one-sided result is recorded as a possible improvement to investigate further.

Failure case: T-shirt/top generation



Figure 7: 64 generated samples conditioned on T-shirt/top, the weakest measured class, arranged in row-major order in an 8x8 grid. The classifier flags 12 images as another class, but these are not marked visually. The notebook lists the five highest-scoring predictions: row 8, column 1 (Shirt, 1.00); row 6, column 1 (Dress, 0.98); row 3, column 8 (Shirt, 0.88); row 1, column 8 (Shirt, 0.87); and row 2, column 5 (Dress, 0.83).

In this sample, the classifier assigned 12 of the 64 images to another class. Its strongest prediction was at row 8, column 1, where it predicted Shirt with a softmax score of 1.00. The next was at row 6, column 1, where it predicted Dress with a score of 0.98.

The classifier also struggled with real T-shirt/top images, correctly classifying 0.81 of them. Shirt was its most common incorrect prediction for both real and generated images: 0.14 of real images and 0.17 of generated images were classified as Shirt. This suggests that some of the confusion is already present when classifying real images.

It is therefore unclear whether the additional errors come from the generator, the classifier, or both. The 12 flagged images show where the classifier disagrees with the conditioning label, but they do not establish that all 12 are generation failures. Figures 4 and 5 provide further visual context for this comparison.

Interpretation for a Non-Technical Audience

The generator is a computer program with one job: when told which of ten clothing types to draw, such as a shirt, a sneaker, or a bag, it creates a small grayscale image of that item. Nobody gave it rules about what each item should look like. Instead, it learned patterns from examples in the Fashion-MNIST dataset, which contains 60,000 training images, with 6,000 examples of each type. Each new image starts from a random point, so the same request can produce different results.

The main results are:

- A separate checking program was used to test the outputs. On real images it had not seen during training, it correctly identified the type about 9 out of 10 times (`0.9074`). On generated images, it recognized the requested type about 9 out of 10 times as well (`0.9008`).
- The generated images have the general shape of the requested clothing type, but they look more like rough sketches than product photographs. They are grainier and show less variation in shades of gray than real images.
- T-shirt/top is the hardest category. The checking program recognizes about 3 out of 4 generated T-shirt/top images (`0.75`). It also struggles with real T-shirt/top images, which suggests that part of the difficulty comes from how similar some clothing types are in the dataset, not only from the generator.
- The model was trained two more times with different random starting points. Scores stayed close (`0.9008`, `0.9020`, and `0.8843`), which suggests the main result does not depend on one particular starting point.

There are three limits to keep in mind:

- **Narrow training data:** The generator learned only from this one dataset of small, grayscale product images, so its understanding of clothing is limited.
- **Low-resolution output:** The generated images are rough, low-resolution pictures, not realistic product photos.
- **Appropriate uses:** The outputs can serve as rough placeholders or illustrations. They should not be used for a product page, a sizing decision, or any situation where the item must be shown accurately.

Ethical Considerations and Responsible Use

Risks of generative image models

Adversarial image generation is one of the techniques behind deepfakes, which can enable impersonation, defamation, and the spread of misleading images.

Although this model cannot produce realistic deepfakes, responsible-use principles still apply to generative image work.

Dataset limitations and responsible use

Built-in ambiguity: the classifier confuses similar clothing types, particularly T-shirt/top and Shirt. This makes it harder to judge how accurately the generated images represent their requested classes, since some errors reflect the dataset itself rather than the generator.

The upstream Fashion-MNIST repository is distributed under the MIT license, as recorded in `DATASET.md`, and this project uses the dataset under those terms.

Reproducibility and transparency

Several project choices support reproducibility and transparency:

- **Seeding and logging:** Fixed seeds and logged losses make training easier to inspect and repeat.
- **Data splits:** The validation and test splits are kept separate; the test split is used only after training and never for model selection.
- **Failure analysis:** The failure analysis is kept in the project artifacts rather than left out of the report.
- **Data verification:** Before training, the notebook verifies the dataset archive checksums instead of relying only on the documentation.
- **Figure checks:** The figures are checked against the outputs recorded by the code.

Limitations and Future Improvements

Model limitations

At 28x28 resolution, the generated samples remain visibly grainy, and their grayscale values are flatter than those in the real images. T-shirt/top is the weakest measured class. Its confusion pattern matches the existing ambiguity between T-shirt/top and Shirt in the dataset. The results, however, do not show whether the higher misclassification rate for generated images comes from the generator, the classifier's decision boundaries, or both.

Evaluation limitations

The agreement metric also has limitations of its own. For instance, it relies on a separate classifier with `421,834` parameters, trained for `10` epochs, whose validation accuracy is printed after each epoch in the notebook. This metric measures agreement with the

requested class, not image realism. With `1,000` tiles per class and an agreement rate near `0.75`, the standard error of a single class estimate is approximately `0.014`. That standard error changes with the measured agreement rate.

Experimental limitations

The classifier is monitored on a held-out validation split, while the test split is evaluated once, after training. Validation results never enter gradient updates or drive model selection, which keeps the test split separate from the monitoring process.

The additional measurements have their own limits:

- **Fréchet distance:** It is computed in this classifier's own feature space, so its values are specific to this project and are not directly comparable with Inception-based results in the literature. A measurable distance also does not prove that the generated images reproduce real-image texture.
- **Diversity ratio:** It is based on `1,000` tiles per class.
- **Run-to-run variation:** The headline results come from one disclosed `50`-epoch run, supported by two further seeds and one ablation run. The class agreement scores from the three seeds have a spread of `0.0177`. This shows the variation observed in these runs but does not establish how much results may vary across other seeds.
- **BatchNorm statistics:** The generated samples depend on the model's BatchNorm running statistics, which are used during evaluation and remain part of the trained model.

The architecture is small by design. The project uses one generator architecture, one main label-smoothing configuration, approximately `2.3` million parameters across the generator and discriminator, and four `50`-epoch runs in total.

Future improvements

- **One-sided label smoothing:** The ablation showed higher agreement and a smaller feature-space Fréchet distance for the one-sided variant. Adopting it would require a fresh headline run trained with one-sided targets.
- **Feature matching:** Matching the discriminator's intermediate feature statistics (Salimans et al., 2016) could stabilize generator training.
- **Auxiliary classifier:** Predicting the class from the discriminator (Odena et al., 2017) could improve label consistency, particularly for T-shirt/top and Shirt.
- **Testing:** Both changes need testing to measure their actual effect.

Longer-term directions:

- **Higher resolution:** Extend training beyond 28x28 images.
- **VAE comparison:** Benchmark against a variational autoencoder.
- **EuroSAT transfer:** Test whether conditional generation works on a non-apparel domain. This is a new task, so it requires its own dataset and class analysis.
- **Responsible use:** Satellite imagery can support monitoring and surveillance, so that extension should define its intended use and limits.

References

Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., & Hochreiter, S. (2017). GANs trained by a two time-scale update rule converge to a local Nash equilibrium. *Advances in Neural Information Processing Systems, 30*. https://papers.nips.cc/paper_files/paper/2017/hash/8a1d694707eb0fefe65871369074926d-Abstract.html

Mirza, M., & Osindero, S. (2014). *Conditional generative adversarial nets*. arXiv. https://arxiv.org/abs/1411.1784

Odena, A., Olah, C., & Shlens, J. (2017). Conditional image synthesis with auxiliary classifier GANs. In *Proceedings of the 34th International Conference on Machine Learning* (pp. 2642-2651). PMLR. https://arxiv.org/abs/1610.09585

Radford, A., Metz, L., & Chintala, S. (2016). *Unsupervised representation learning with deep convolutional generative adversarial networks*. arXiv. https://arxiv.org/abs/1511.06434

Salimans, T., Goodfellow, I., Zaremba, W., Cheung, V., Radford, A., & Chen, X. (2016). Improved techniques for training GANs. *Advances in Neural Information Processing Systems, 29*. https://proceedings.neurips.cc/paper_files/paper/2016/hash/8a3363abe792db2d8761d6403605aeb7-Abstract.html

Xiao, H., Rasul, K., & Vollgraf, R. (2017). *Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms*. arXiv. https://arxiv.org/abs/1708.07747